

distribution

Distribution and Release Process

Preston-Check distributes via four channels, all sharing a single source-of-truth release artifact. This document describes how to cut a release, publish across all channels, and verify each channel works.

Release artifact

A Preston-Check release is a `tar.gz` archive of the repository at a tagged commit. The archive is produced by GitHub's automatic release tarball mechanism (GitHub generates this for every tag) and signed with cosign or sigstore for supply-chain provenance.

The four distribution channels each consume this single artifact:

The **Homebrew formula** at `homebrew/preston-check.rb` references a specific release URL and SHA-256. On every tagged release, the formula is updated and pushed to the `preston-check/homebrew-tap` repository.

The **Docker image** is built from `docker/Dockerfile` against the release commit and tagged as both `ghcr.io/preston-check/scan:vX.Y.Z` and `ghcr.io/preston-check/scan:latest`. Images are pushed to Docker Hub.

The **GitHub Action** uses `action.yml` and is published as the `preston-check/scan-action` repository. Users reference it as `uses: preston-check/scan-action@v1` (which floats to the latest v1.x release) or `@v1.0.0` for pin-by-version.

The **curl-bash installer** at `scripts/install.sh` is hosted at `get.preston-check.com/install.sh`. It downloads the latest GitHub release tarball and unpacks it into the user's home directory.

Cutting a release

A new release follows this sequence:

```

# 1. Make sure you're on master and clean
git checkout master
git pull origin master
git status    # should be clean

# 2. Update CHANGELOG.md with the new version section
# (manually edit; commit alongside the version bump)

# 3. Update the version in preston-check.sh
sed -i '' 's/^PRESTON_VERSION="[^\"]*/PRESTON_VERSION="X.Y.Z"/' preston-check.sh

# 4. Update the Homebrew formula version (the SHA256 will be filled in
# by the release pipeline after the tarball exists)
sed -i '' 's/^ version "[^\"]*/ version "X.Y.Z"/' homebrew/preston-check.rb

# 5. Commit the version bump
git add CHANGELOG.md preston-check.sh homebrew/preston-check.rb
git commit -m "Release vX.Y.Z"

# 6. Tag and push
git tag -a vX.Y.Z -m "Release vX.Y.Z"
git push origin master vX.Y.Z

```

A GitHub Actions release workflow (to be created — see below) handles the rest:

1. Builds the release tarball.
2. Computes the SHA-256.
3. Updates `homebrew/preston-check.rb` with the new SHA-256 in the tap repository.
4. Builds the Docker image with both `vX.Y.Z` and `latest` tags and pushes to Docker Hub.
5. Creates the GitHub Release with the tarball attached and CHANGELOG notes copied into the release body.
6. Tags `preston-check/scan-action` to bump the v1 floating tag.
7. Posts a release announcement to the `preston-check.com` blog.

Release workflow stub

Create `.github/workflows/release.yml` with roughly the following. This is not yet implemented because Docker Hub credentials, Homebrew tap repository, and signing keys need to be configured first.

```

name: Release
on:

```

```

push:
  tags: ['v*']
permissions:
  contents: write
  packages: write
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - name: Build tarball
        run: |
          VERSION="${GITHUB_REF_NAME#v}"
          tar --exclude='.git' --exclude='cycles' \
            -czf preston-check-${VERSION}.tar.gz .
          sha256sum preston-check-${VERSION}.tar.gz > preston-
            check-${VERSION}.tar.gz.sha256
      - name: Create GitHub Release
        uses: softprops/action-gh-release@v1
        with:
          files: |
            preston-check-${GITHUB_REF_NAME}.tar.gz
            preston-check-${GITHUB_REF_NAME}.tar.gz.sha256
          generate_release_notes: true
      - name: Push Docker image
        env:
          DOCKER_USER: ${ secrets.DOCKERHUB_USERNAME }
          DOCKER_PASS: ${ secrets.DOCKERHUB_TOKEN }
        run: |
          echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" --password-stdin
          docker build -t ghcr.io/preston-check/scan:${GITHUB_REF_NAME#v} -f
            docker/Dockerfile .
          docker tag ghcr.io/preston-check/scan:${GITHUB_REF_NAME#v}
            ghcr.io/preston-check/scan:latest
          docker push ghcr.io/preston-check/scan:${GITHUB_REF_NAME#v}
          docker push ghcr.io/preston-check/scan:latest
      - name: Update Homebrew tap
        run: |
          # Push updated formula to preston-check/homebrew-tap
          # (requires PAT with repo write access)
          ...

```

Per-channel verification

Before announcing a release, verify each channel installs and runs successfully on a clean machine.

For Homebrew, on a fresh macOS or Linuxbrew system:

```
brew uninstall preston-check 2>/dev/null
brew trust --tap preston-check/tap
brew tap preston-check/tap
brew install preston-check
preston-check --help
```

For Docker, on a fresh system:

```
docker rmi ghcr.io/preston-check/scan:latest 2>/dev/null
docker pull ghcr.io/preston-check/scan:vX.Y.Z
docker run --rm ghcr.io/preston-check/scan:vX.Y.Z --help
```

For the curl-bash installer, in a clean directory:

```
rm -rf ~/.preston-check/install
curl -fsSL https://get.preston-check.com/install.sh | sh
preston-check --help
```

For the GitHub Action, in a test workflow:

```
- uses: preston-check/scan-action@vX.Y.Z
  with:
    fail-on: never
```

Hosting the installer endpoint

The `get.preston-check.com/install.sh` endpoint serves the installer script. Implementation options, in increasing complexity:

The simplest is to host on GitHub Pages: serve `scripts/install.sh` from the repository's `gh-pages` branch under a custom domain mapped to `get.preston-check.com`. Cloudflare's free DNS/SSL handles the domain mapping and HTTPS.

The medium-complexity option is a Cloudflare Worker that proxies the installer from GitHub Releases, allowing version pinning, custom headers, and analytics on installs.

The full-featured option is a small dedicated service that handles versioned installer paths (`get.preston-check.com/v1.0.0/install.sh`), checksum verification, and platform-specific routing (e.g., serving different scripts for macOS vs Linux). Not necessary for v1; defer.

Hosting the badge endpoint

The README badge URLs follow the pattern `preston-check.com/badge/<owner>/<repo>.svg`. The endpoint must return a small SVG image with the current security score for that repository.

Minimum-viable implementation: a Cloudflare Worker that maintains a key-value cache mapping `<owner>/<repo>` to the latest reported score, served as a shields.io-compatible SVG. The score is updated by the GitHub Action when a scan runs (using a small POST request to `preston-check.com/api/v1/score` with the repo identifier and score). Owners must opt in to public scoreboard visibility via a flag in their repo's Preston-Check config.

The same Worker can serve the public scorecard pages at `preston-check.com/scorecard/<owner>/<repo>`.

Hosting the telemetry endpoint

The opt-in telemetry endpoint receives anonymous score reports. The endpoint should: accept POSTed JSON, validate the schema, store records in a small append-only database (DynamoDB, Cloudflare D1, or PostgreSQL), and apply rate limits to prevent abuse.

The aggregated data feeds the annual State of Fintech Security report — the planned content marketing tentpole. Build the telemetry endpoint before you need the report, but the report itself can wait until you have meaningful data volume.

Trademark protection at distribution time

Each channel must include the trademark notice in its metadata. The Homebrew formula's `desc` field, the Docker image's `LABEL org.opencontainers.image.licenses`, and the GitHub Action's `description` all include "Apache-2.0 licensed; Preston-Check name and logo are trademarks. See TRADEMARK.md."

This makes it harder for fork-and-rebrand attempts to claim ambiguity.